NOTE TECHNIQUE INTERNE — DOCUMENT COMPLÉMENTAIRE AU RAPPORT

Installation, environnement et structure du projet

À l'attention de l'encadrant — Raspberry Pi 4 & Jetson Nano (JN30D)

Ce document récapitule, de façon pratique, l'ensemble des commandes exécutées, des paquets téléchargés/installés et de la structure de fichiers mise en place sur les deux cartes utilisées durant le stage. Il complète le rapport en donnant une vue directement exploitable de l'environnement technique réel du projet.

⚠️ *Les commandes ci-dessous reconstituent la procédure suivie à partir des éléments techniques validés du projet. Merci de les relire et de les ajuster si un chemin ou un nom diffère légèrement de ce qui a été réellement tapé sur le terminal avant de transmettre ce document.*

1. Raspberry Pi 4

1.1 Préparation de la carte SD

Le système a été installé via l'outil Raspberry Pi Imager (sur le PC), avec l'image « Raspberry Pi OS (64-bit) » basée sur Debian Bookworm. Dans les options avancées de l'Imager (icône ⚙️), les paramètres suivants ont été pré-configurés avant le flashage : nom d'hôte, activation du SSH, identifiants utilisateur, et connexion Wi-Fi — ce qui permet un premier démarrage headless (sans écran ni clavier).

1.2 Connexion à la carte

```
ssh pi@<adresse_ip_raspberry>
```

1.3 Environnement Python — gesture_env

Un environnement virtuel Python dédié, gesture_env, isole les dépendances du projet du système global :

```
python3 -m venv ~/gesture_env
source ~/gesture_env/bin/activate

# mise à jour de pip avant installation des paquets
pip install --upgrade pip
```

Une fois activé, l'invite de commande affiche le préfixe (gesture_env) confirmant que l'environnement est bien actif. Il doit être réactivé (source ~/gesture_env/bin/activate) à chaque nouvelle session SSH avant de lancer le pipeline.

1.4 Bibliothèques installées

```
pip install opencv-python==4.10.0.84
pip install mediapipe==0.10.18
pip install numpy==1.26.4
pip install ai-edge-litert
pip install flask
```

Rôle de chaque paquet :

- opencv-python — capture vidéo, traitement d'image, affichage des overlays (zones, repères, texte) sur le flux.
- mediapipe — détection et suivi des points de la main (landmarks) en temps réel.
- numpy — manipulation des images sous forme de tableaux numériques.
- ai-edge-litert — exécution du modèle EfficientDet-Lite0 (.tflite).
- flask — serveur web du tableau de bord (flux MJPEG + statistiques de cycle).

1.5 Modèles téléchargés

- efficientdet_lite0.tflite — modèle pré-entraîné de détection d'objets (condition de démarrage de cycle), placé dans models/.
- gesture_classifier_mobilenetv2.tflite — classifieur de gestes (MobileNetV2, ~2 167 images / 8 classes) téléchargé pour une intégration future, non encore branché au pipeline principal.

1.6 Structure des fichiers du projet

```
~/projet_gestion_cycles/
├── scripts/
│   ├── app.py                  # serveur Flask + tableau de bord
│   ├── main_pipeline.py         # boucle principale (capture, machine à
états)
│   ├── state_machine.py         # logique IDLE / PIECE_DETECTEE /
EN_COURS_TRAVAIL / TERMINE
│   └── templates/
│       └── dashboard.html       # page web du tableau de bord
├── models/
│   ├── efficientdet_lite0.tflite
│   └── gesture_classifier_mobilenetv2.tflite
├── logs/
│   └── session_<horodatage>.csv # un fichier de log par lancement
└── gesture_env/                 # environnement virtuel Python
```

1.7 Lancement du système

```
cd ~/projet_gestion_cycles
source gesture_env/bin/activate
python scripts/app.py

# tableau de bord accessible depuis le réseau local :
# http://<adresse_ip_raspberry>:5000
```

2. Jetson Nano (Auvidea JN30D)

2.1 Image système

La carte utilise le BSP fourni par Auvidea (et non l'image JetPack standard NVIDIA), indispensable pour la reconnaissance correcte des périphériques du carrier board JN30D. Le système résultant correspond à L4T R32.7.1 / JetPack 4.6.x, sur base Ubuntu 18.04, architecture aarch64.

2.2 Connexion à la carte

PC (Windows, partage de connexion ICS) — 192.168.137.1 ⟷ Jetson Nano — 192.168.137.50, en Ethernet direct.

```
# côté Jetson : attribution d'une IP statique
sudo nmcli con add type ethernet ifname eth0 con-name jetson-eth \
  ip4 192.168.137.50/24 gw4 192.168.137.1
sudo nmcli con up jetson-eth

# côté PC : connexion via PuTTY
putty.exe -ssh <utilisateur>@192.168.137.50
```

⚠️ *Sur le PC Windows : la case « Partage de connexion (ICS) » se désactive automatiquement après veille/redémarrage et doit être réactivée manuellement à chaque session. La gestion d'énergie de la carte réseau doit être désactivée pour éviter une coupure SSH après ~1h.*

2.3 Installation de Miniforge (conda)

```
wget https://github.com/conda-forge/miniforge/releases/latest/download/
Miniforge3-Linux-aarch64.sh
bash Miniforge3-Linux-aarch64.sh
source ~/.bashrc
```

2.4 Création de l'environnement gesture_env

```
conda create -n gesture_env python=3.11
conda activate gesture_env
```

2.5 Installation des bibliothèques (versions figées)

```
pip install mediapipe==0.10.18
pip install numpy==1.26.4
pip install opencv-python==4.10.0.84
pip install ai-edge-litert
pip install flask
```

Ces versions sont figées volontairement : mediapipe 0.10.18 est la dernière version disposant d'un paquet précompilé pour ARM64 ; numpy 1.26.4 évite l'incompatibilité de numpy 2.x avec la version de GLIBCXX disponible sur Ubuntu 18.04 (base de L4T) ; opencv-python 4.10.0.84 évite la dépendance à numpy 2.x introduite à partir d'OpenCV 5.x.

2.6 Correctifs système (hooks d'activation conda)

Deux variables d'environnement doivent être positionnées automatiquement à chaque activation de gesture_env, via des scripts placés dans le dossier d'activation de l'environnement conda :

```
mkdir -p $CONDA_PREFIX/etc/conda/activate.d
mkdir -p $CONDA_PREFIX/etc/conda/deactivate.d
```

```
cat > $CONDA_PREFIX/etc/conda/activate.d/env_vars.sh << 'EOF'
#!/bin/bash
# fournit un libstdc++ suffisamment récent (symbole GLIBCXX_3.4.29)
export OLD_LD_LIBRARY_PATH=$LD_LIBRARY_PATH
export LD_LIBRARY_PATH=$CONDA_PREFIX/lib:$LD_LIBRARY_PATH

# corrige l'erreur 'static TLS' de libgomp sur aarch64
export OLD_LD_PRELOAD=$LD_PRELOAD
export LD_PRELOAD=/usr/lib/aarch64-linux-gnu/libgomp.so.1
EOF

cat > $CONDA_PREFIX/etc/conda/deactivate.d/env_vars.sh << 'EOF'
#!/bin/bash
export LD_LIBRARY_PATH=$OLD_LD_LIBRARY_PATH
unset OLD_LD_LIBRARY_PATH
export LD_PRELOAD=$OLD_LD_PRELOAD
unset OLD_LD_PRELOAD
EOF
```

⚠️ *Les chemins exacts (libgomp.so.1, etc.) peuvent varier légèrement selon l'installation : à vérifier avec find / -name "libgomp.so.1" si besoin avant de documenter la version finale.*

2.7 Vérification de la caméra

```
ls /dev/video*
v4l2-ctl --list-devices

# la caméra USB doit être branchée sur un port direct de la carte,
# pas sur un hub USB (instabilité constatée sinon)
```

2.8 Structure des fichiers du projet

```
~/projet_gestion_cycles/           # transféré depuis le PC via FileZilla
(SFTP)
├── scripts/
│   ├── app.py
│   ├── main_pipeline.py
│   ├── state_machine.py
│   └── templates/
│       └── dashboard.html
├── models/
│   ├── efficientdet_lite0.tflite ->
~/models_partages/efficientdet_lite0.tflite   # symlink
│   └── gesture_classifier_mobilenetv2.tflite
└── logs/
    └── session_<horodatage>.csv
```

2.9 Lancement du système

```
conda activate gesture_env
cd ~/projet_gestion_cycles
python scripts/app.py

# tableau de bord accessible depuis le PC Windows :
# http://192.168.137.50:5000
```

3. Récapitulatif — éléments téléchargés / installés

Élément	Raspberry Pi 4	Jetson Nano (JN30D)
Image système	Raspberry Pi OS Bookworm 64-bit (Raspberry Pi Imager)	BSP Auvidea (L4T R32.7.1 / JetPack 4.6.x)
Gestion d'environnement	venv (gesture_env)	Miniforge / conda (gesture_env, Python 3.11)
opencv-python	4.10.0.84	4.10.0.84
mediapipe	0.10.18	0.10.18
numpy	1.26.4	1.26.4
ai-edge-litert	dernière version	dernière version
flask	dernière version	dernière version
Modèle EfficientDet-Lite0	models/efficientdet_lite0.tflite	models/efficientdet_lite0.tflite (symlink)
Modèle MobileNetV2 (gestes)	gesture_classifier_mobilenetv2.tflite (non intégré)	gesture_classifier_mobilenetv2.tflite (non intégré)
Transfert de fichiers	FileZilla (SFTP)	FileZilla (SFTP)
Accès distant	SSH direct (réseau local)	PuTTY (SSH) via Ethernet + ICS

Ce document, avec le chapitre du rapport correspondant, permet de reproduire intégralement l'environnement de travail sur les deux plateformes.